Paper 2 — How Deep Linear Networks Learn

Andrew M. Saxe, James L. McClelland, Surya Ganguli (ICLR 2014)

Exact Solutions to the Nonlinear Dynamics of Learning in Deep Linear Neural Networks | RQ2: Training dynamics; RQ4: theory vs. practice

The paper in one minute

Question: Why can learning be slow in deep networks even if every layer is linear? **Model:** Multiple linear weight matrices multiplied together. **Finding:** Their combined input–output function is linear, but the *weight updates* are nonlinear. Training can show long plateaus, then rapid learning; depth and initialization change this behavior. **Why we care:** A training plateau does not necessarily mean a bad local minimum.

1. Start with the network, not the differential equations

For a two-layer network with input x, hidden representation h and output $\hat{y}$, write $h = w_1 x$, then $\hat{y} = w_2 h = w_2 w_1 x$. There is *no activation function*. The input–output map is linear in x, but the optimizer must learn w_1 and w_2 *separately*; their product creates non-convex training dynamics. With $x = y = 1$, let

$$L(w_1, w_2) = \tfrac{1}{2}(1 - w_2 w_1)^2.$$

Gradient recap A partial derivative measures how loss changes as *one* weight changes while the other stays fixed. Gradient descent uses $w_i \leftarrow w_i - \eta\, \partial L/\partial w_i$, where $\eta > 0$ is the learning rate. By the chain rule,

$$\frac{\partial L}{\partial w_1} = -(1 - w_1 w_2) w_2, \qquad \frac{\partial L}{\partial w_2} = -(1 - w_1 w_2) w_1.$$

So the updates are

$$\boxed{w_1^+ = w_1 + \eta(1 - w_1 w_2) w_2, \qquad w_2^+ = w_2 + \eta(1 - w_1 w_2) w_1.}$$

Why this matters in ML

The update for w_1 depends on w_2, and vice versa: the weights are **coupled**. With $w_1 = w_2 = 0.01$, the prediction is 0.0001 and the loss is about 0.5, but each gradient has magnitude only ≈ 0.01. Thus *large error does not imply a large gradient*. Small weights can mean slow initial progress.

2. Depth makes the multiplicative effect clearer

For d scalar layers, $\hat{y} = (w_d \cdots w_2 w_1) x$. At any given layer i, differentiating the squared error gives

$$\frac{\partial L}{\partial w_i} = (\hat{y} - y) x \prod_{j \neq i} w_j.$$

If the other weights are all tiny, their product may be extremely tiny; if too large, the product may be huge. This is one mechanism behind very slow or unstable learning. The full paper goes beyond this toy example by analyzing many-dimensional linear networks and the learning of individual data modes.

Important precision

The authors do *not* claim every plateau is a saddle, or that deeper networks always take exponentially longer. Their exact results reveal special initializations for which learning times need not blow up with depth.

3. Gradient flow: from steps to a continuous curve

We start with the two-weight loss $L(a,b) = \frac{1}{2}(1-ab)^2$. Ordinary gradient descent changes each weight in discrete steps: $a_{k+1} = a_k - \eta\,\partial L/\partial a$, where η is the learning rate. Rearranging gives $(a_{k+1} - a_k)/\eta = -\partial L/\partial a$. As $\eta \to 0$ and continuous time is defined by $t = k\eta$, the change per unit time becomes a derivative:

$$\boxed{\frac{da}{dt} = -\frac{\partial L}{\partial a}, \qquad \frac{db}{dt} = -\frac{\partial L}{\partial b}.}$$

Intuition. Gradient descent asks *how much will the weight change in the next step?* Gradient flow asks *how fast is it changing right now?* This continuous-time approximation makes learning mathematically easier to study; finite-step descent does not necessarily follow exactly the same path.

4. Worked example: why learning follows an S-shaped curve

Step 1 — Differentiate one weight at a time. Holding b fixed while differentiating a (and vice versa), the chain rule gives

$$\frac{\partial L}{\partial a} = -b(1-ab), \quad \frac{\partial L}{\partial b} = -a(1-ab) \quad \Longrightarrow \quad \frac{da}{dt} = b(1-ab),\ \frac{db}{dt} = a(1-ab).$$

The weights are **coupled**: the update of each depends on the other.

Step 2 — Why assume $a = b = w$? If we initialize both layers with equal weights, their update equations are identical, so they remain equal. It is a useful *special case*, not a statement about arbitrary initialization. Substitute $a = b = w$ into either original layer's equation:

$$\boxed{\frac{dw}{dt} = w(1-w^2).}$$

Step 3 — Read the equation. At $w = 0.01$, the rate is approximately 0.01 (slow despite large error); at $w = 0.5$, it is 0.375 (faster); and at $w = 1$, it is zero (the prediction $ab = 1$ matches the target). Learning speed changes over time.

Step 4 — Track the whole network. The effective weight, and our prediction for $x = 1$, is $u = ab = w^2$. This u is a *scalar*, not the PCA direction vector from Paper 1. By the chain rule,

$$\frac{du}{dt} = \frac{d(w^2)}{dt} = 2w\frac{dw}{dt} = 2w^2(1-w^2) \quad \Longrightarrow \quad \boxed{\frac{du}{dt} = 2u(1-u).}$$

Step 5 — Interpret the result. For $0 < u < 1$, the rate $2u(1-u)$ is small near $u = 0$, largest in the middle, and small again near $u = 1$. Hence the effective prediction grows in an **S-shaped curve**: slow start → rapid learning → leveling off. The loss $L = \frac{1}{2}(1-u)^2$ shows an initial plateau followed by a rapid decrease. The exact solution for $0 < u_0 < 1$ is

$$u(t) = \frac{1}{1 + \left(\frac{1-u_0}{u_0}\right)e^{-2t}}.$$

A smaller positive u_0 means a longer early plateau. This derivation is a *teaching example*, not the paper's general multivariate theorem.

Important: why there is no extra factor of 2

Differentiate $L(a,b)$ with respect to one layer's weight *before* setting $a = b = w$. If we instead differentiate the reduced loss $\frac{1}{2}(1-w^2)^2$ directly, that changes *both* weights at once and produces an extra factor of 2. These are different derivatives.

5. From the example to Saxe et al.'s paper

A general deep linear network computes $\hat{\mathbf{y}} = W_d \cdots W_2 W_1 \mathbf{x}$. The authors use singular-value structure (recall SVD: $M = U\Sigma V^T$) to study how different *data modes* learn at different speeds. Special initialization choices can avoid large depth-dependent learning delays in the settings they analyze.

Connection to Paper 1 and our survey

Baldi–Hornik asks *where are the minima and saddles?* Saxe et al. asks *how do the weights move toward a solution?* A plateau does not necessarily mean the network is trapped at a bad local minimum: coupled weight dynamics and initialization alone may make progress very slow.

5. Initialization and the depth problem

Suppose the weights are initialized randomly. If many successive matrices shrink the input or backward gradient, learning signals may become extremely small. If they amplify them, signals may grow too large. It is therefore not enough to ask what the best solution looks like; **how the network starts** can change how quickly it gets there.

Orthogonal initialization from first principles A square orthogonal matrix Q obeys $Q^TQ = I$ (for compatible rectangular cases, the relevant orthogonality condition changes). Since $\|Qx\|_2^2 = (Qx)^T(Qx) = x^TQ^TQx = x^Tx$, an orthogonal transformation preserves vector length. Hence it can help preserve signal magnitude as information propagates through layers rather than repeatedly shrinking or stretching it.

The main paper result

Saxe et al. derive exact learning dynamics for deep linear networks and show that special initial conditions can yield **finite, depth-independent delays** relative to shallow networks even at large depth. They analyze how suitable unsupervised pretraining and random orthogonal initializations can realize such favorable behavior under specified conditions. This is more nuanced than saying "depth always slows learning."

6. What this paper actually contributes

1. **Exact dynamics:** mathematically tractable descriptions of how deep linear network weights and data modes evolve under learning.
2. **Plateau–transition behavior:** a way to explain long intervals of little apparent progress followed by rapid learning, despite linear input–output computation.
3. **Depth & initialization:** conditions under which very deep networks can learn without an ever-growing penalty from depth, motivating carefully structured initialization.
4. **Bridge to practice:** insights into learning dynamics also seen empirically in nonlinear networks, without claiming a full theory of modern deep-learning optimization.

7. Assumptions, limitations, and survey connection

Assumptions. The exact theory focuses on deep *linear* architectures, squared-error learning, and particular data structures, initial conditions, and dynamical regimes; matrix-mode decoupling requires appropriate conditions. The paper also discusses special nonlinear-network signal-propagation regimes, but those do not make its exact linear results universal.

Limitations. It does not establish a universal convergence rate for arbitrary ReLU networks, stochastic minibatch SGD, or saddle-escape algorithms. A long plateau need not be a saddle point, and orthogonal initialization is not a guarantee of successful learning in every setting.

One-paragraph takeaway for our notes

Even with no nonlinear activations, a deep network can learn in a really nonlinear way because its weight matrices are multiplied together. Small or poorly initialized weights can make the gradients tiny, so training seems stuck before it suddenly picks up. The interesting part is that this doesn't necessarily mean the loss has a bad local minimum: how we initialize the network, and which data directions it's learning, can make a huge difference to training speed.

8. Quick reference

Concept	Meaning / equation
Deep linear model	$\hat{\mathbf{y}} = W_d \cdots W_1 \mathbf{x}$; linear in input, nonlinear in separate weights.
Gradient flow	$dW_i/dt = -\nabla_{W_i} L$; continuous-time gradient descent.
Effective scalar weight	$u = w_1 w_2$; for symmetric two-layer case $du/dt = 2u(1-u)$.
SVD	$M = U\Sigma V^T$; separates directions and their strengths.
Orthogonality	$Q^TQ = I$; preserves lengths and can aid signal propagation.

Source:

Saxe, A. M., McClelland, J. L., & Ganguli, S. (2014). *Exact Solutions to the Nonlinear Dynamics of Learning in Deep Linear Neural Networks.* ICLR. https://www.saxelab.org/assets/papers/Saxe2014.pdf. Scalar examples and explanatory wording are pedagogical adaptations.